

Smart Home Security System

Project Documentation

Using ESP32 Microcontroller & Blynk IoT Platform

Technical Overview & Metadata

Developer	Chitturi Sanjay Kumar
Affiliation	IEEE Student Member
Platform	ESP32 Core Framework + Blynk Cloud Platform
Domain	Internet of Things (IoT) / Embedded Systems Design
GitHub	github.com/Sanjaykumar9441
Simulation	Wokwi Online Virtual Laboratory
Language	Embedded C / C++ (Arduino Core Architecture)

Open-source System Architecture — Released for Educational and Research Frameworks.

Table of Contents

Contents

1	Abstract	4
2	Introduction	4
3	Project Objectives	4
4	System Architecture	5
4.1	Perception Layer	5
4.2	Processing Layer	5
4.3	Application Layer	6
5	Hardware Components	6
6	Circuit & Pin Configuration	7
7	Blynk IoT Configuration	8
7.1	Push Notification Events	9
8	Software & Tools	10
9	System Operation	10
9.1	Normal Operating Condition	10
9.2	Alert Thresholds	10
9.3	Emergency Response Sequence	11
9.4	System Flow Diagram	11
10	Project Structure	12
11	Setup & Deployment Guide	12
11.1	Prerequisites	12
11.2	Library Installation	13
11.3	Firmware Configuration	13
11.4	Flashing the ESP32	13
11.5	Wokwi Simulation	13
12	Results & Observations	14

13 Applications	14
14 Future Enhancements	15
15 Conclusion	15
References	16

1. Abstract

The **Smart Home Security System** is a real-time IoT-based embedded solution designed to continuously monitor the home environment for potential hazards and unauthorized intrusion. Built around the **ESP32** microcontroller and integrated with the **Blynk IoT** cloud platform, the system employs four sensors — DHT22 (temperature and humidity), MQ2 (gas leakage), a flame sensor (fire detection), and a PIR motion sensor (human intrusion) — to provide comprehensive safety coverage.

Upon detecting any hazardous condition, the system autonomously triggers an alarm buzzer and activates a relay module while simultaneously pushing real-time notifications to the user's smartphone through the Blynk application. A live dashboard provides continuous visibility into all sensor readings, making this system suitable for smart home automation, embedded systems coursework, and IEEE project demonstrations.

2. Introduction

The rapid advancement of Internet of Things (IoT) technology has enabled the development of intelligent home automation systems that can perceive, analyze, and respond to environmental conditions in real time. Home security remains one of the most critical applications of IoT, where timely detection and automated response to fire, gas leakage, or unauthorized entry can prevent property damage and protect human lives.

This project implements a low-cost, microcontroller-based smart home security system that addresses four primary safety concerns: environmental temperature extremes, combustible gas leakage, open flame events, and physical intrusion. The system operates autonomously without requiring manual monitoring, leveraging cloud connectivity to deliver real-time alerts regardless of the user's location.

The **ESP32** was chosen as the core controller for its dual-core processing capability, built-in Wi-Fi and Bluetooth, extensive GPIO availability, and broad support within the Arduino ecosystem. **Blynk IoT** serves as the cloud backend and mobile dashboard, providing a user-friendly interface with minimal backend development overhead.

3. Project Objectives

The primary objectives of this project are:

- Monitor ambient temperature and humidity levels continuously in real time.

- Detect the presence of LPG, methane, butane, and smoke gases using the MQ2 sensor.
- Detect open flames and fire events using an infrared flame sensor.
- Detect unauthorized human movement or intrusion using a passive infrared (PIR) sensor.
- Automatically activate a buzzer alarm and relay module upon detection of any hazardous condition.
- Transmit all sensor data to the Blynk IoT cloud platform and update the dashboard in real time.
- Send instant push notifications to the user's smartphone for each alert category.
- Log all emergency events to the Blynk Cloud for historical review.
- Demonstrate the practical application of ESP32 and cloud-based IoT in a safety-critical context.

4. System Architecture

The system follows a layered IoT architecture comprising three primary tiers: the **Perception Layer** (sensors), the **Processing Layer** (ESP32 firmware), and the **Application Layer** (Blynk Cloud and mobile dashboard).

4.1 Perception Layer

The perception layer consists of four sensors that translate physical phenomena into electrical signals readable by the ESP32. The DHT22 provides calibrated digital output for temperature and humidity. The MQ2 provides an analog voltage proportional to gas concentration, read via the ESP32's 12-bit ADC on GPIO 35. The flame sensor and PIR sensor output digital HIGH/LOW signals based on infrared detection.

4.2 Processing Layer

The ESP32 firmware, written in C/C++ using the Arduino framework, performs all data acquisition, threshold evaluation, actuator control, and cloud communication. The main loop reads each sensor at a defined interval, compares readings against configured thresholds, and dispatches alert routines as needed. The Blynk library abstracts Wi-Fi connectivity and virtual pin communication.

4.3 Application Layer

The Blynk IoT platform acts as the cloud broker, receiving sensor data via virtual pins and rendering it on a configurable mobile dashboard. Users receive push notifications through the Blynk mobile application (iOS/Android) whenever a threshold is breached. All events are logged in the Blynk Cloud event stream for audit and review.

5. Hardware Components

Table 1 lists all hardware components used in this project.

Table 1: Hardware Components

Component	Qty	Description
ESP32 Development Board	1	Dual-core 240 MHz processor with integrated Wi-Fi & Bluetooth; acts as the central controller
DHT22 Sensor	1	Capacitive digital temperature & humidity sensor; accuracy $\pm 0.5\text{ }^{\circ}\text{C}$, $\pm 2\%$ RH
MQ2 Gas Sensor	1	Electrochemical sensor detecting LPG, smoke, methane, and butane via analog output
PIR Motion Sensor	1	Passive infrared sensor detecting infrared radiation changes caused by human movement
Flame Sensor	1	Infrared-based sensor detecting wavelengths emitted by open flames (760–1100 nm)
Relay Module	1	Single-channel 5 V relay for switching external loads or cutting power in emergencies
Buzzer	1	Active piezoelectric buzzer for audible alarm output
LED	1	Visual status indicator for system state feedback
220 Ω Resistor	1	Current-limiting resistor for the LED
10 k Ω Pull-up Resistor	1	Required on DHT22 data line for reliable digital communication
Breadboard	1	Prototyping platform for circuit assembly without soldering
Jumper Wires	—	Connections between ESP32, sensors, and actuators

6. Circuit & Pin Configuration

All sensors and actuators connect to the ESP32 as specified in Table 2. GPIO 35 is used for the MQ2 analog output as it is an input-only pin ideal for ADC operation. A 10 k Ω pull-up resistor must be placed between the DHT22 data line and VCC (3.3 V) for reliable

one-wire communication.

Table 2: ESP32 Pin Configuration

Component	ESP32 GPIO	Signal Type
DHT22 Data	GPIO 27	Digital (One-Wire)
PIR Motion Sensor	GPIO 15	Digital Output
Flame Sensor	GPIO 4	Digital Output
MQ2 Gas Sensor (Analog)	GPIO 35	Analog (ADC1_CH7)
Relay Module	GPIO 2	Digital Output
Buzzer	GPIO 14	Digital Output

Note: All sensors are powered from the ESP32's 3.3 V and GND pins. The relay module may require an external 5 V supply depending on the coil voltage specification. A flyback diode across the relay coil is recommended to suppress inductive voltage spikes.

7. Blynk IoT Configuration

The Blynk IoT platform is configured with six virtual pins mapping sensor readings and actuator status to dashboard widgets (Table 3).

Table 3: Blynk Virtual Pin Mapping

Virtual Pin	Widget	Purpose
V0	Gauge	Live temperature reading (°C)
V1	Gauge	Live humidity reading (%)
V2	Gauge	MQ2 gas concentration (ADC raw value, 0–4095)
V3	LED	PIR motion detection status (ON = motion detected)
V4	LED	Relay activation status (ON = relay active)
V5	LED	Flame detection status (ON = flame detected)

7.1 Push Notification Events

The system dispatches the following notifications through the Blynk Event system:

- **Over-Temperature Detected** — triggered when temperature exceeds 50 °C
- **Gas Leakage Detected** — triggered when MQ2 ADC value exceeds 3800
- **Fire Detected** — triggered when the flame sensor output goes LOW
- **Motion Detected** — triggered when the PIR sensor output goes HIGH

8. Software & Tools

Table 4: Software Stack

Tool / Platform	Purpose
Arduino IDE	Primary development environment for writing and uploading ESP32 firmware
ESP32 Board Package	Arduino IDE add-on providing ESP32 hardware support and peripheral drivers
Blynk IoT Platform	Cloud platform for sensor data storage, dashboard rendering, and push notifications
Blynk Library	Arduino library abstracting Blynk API calls and virtual pin communication
DHTesp Library	Arduino library for DHT11/DHT22 sensor communication on ESP32
Wokwi Simulator	Online browser-based circuit and code simulator supporting ESP32 and Blynk
Git & GitHub	Version control and source code hosting
C / C++	Programming language used for all firmware development

9. System Operation

9.1 Normal Operating Condition

Under normal conditions, all sensor readings remain within the safe threshold range. The ESP32 continuously polls each sensor, uploads the readings to Blynk Cloud, and refreshes the dashboard. The buzzer and relay remain in the OFF state. The system operates in this state indefinitely until a threshold is breached.

9.2 Alert Thresholds

An emergency response is triggered when any of the following conditions is met:

Table 5: Alert Thresholds

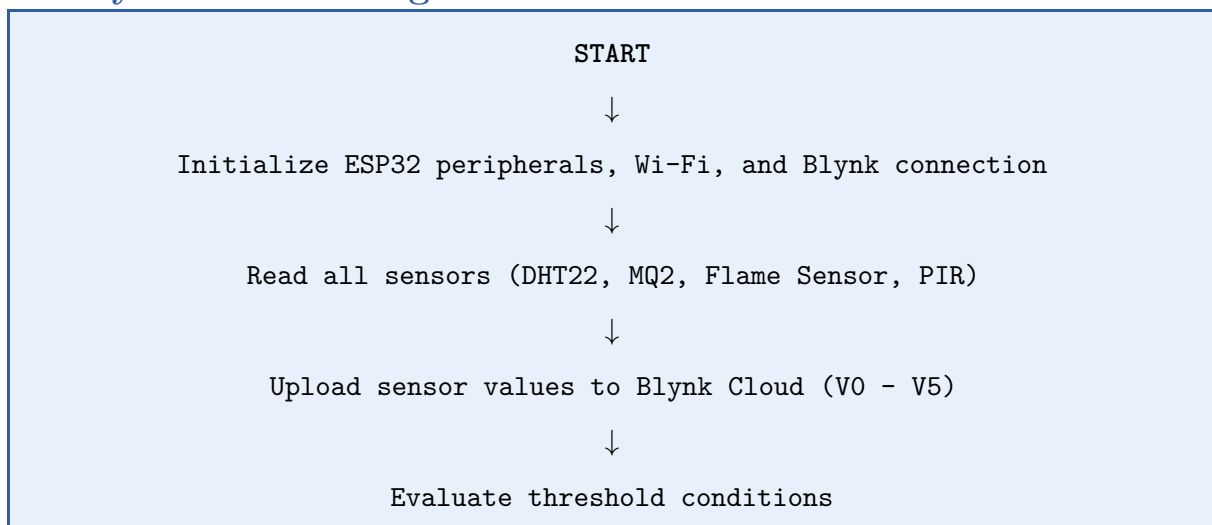
Sensor	Alert Condition	Threshold
DHT22	High Temperature	Temperature > 50 °C
MQ2 Gas Sensor	Gas Leakage	ADC Value > 3800
Flame Sensor	Fire Detected	Sensor Output LOW
PIR Motion Sensor	Intrusion	Sensor Output HIGH

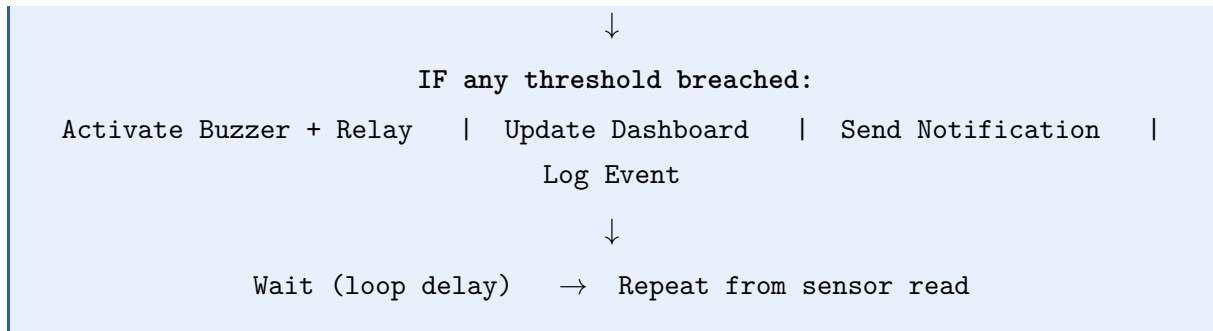
9.3 Emergency Response Sequence

When a threshold breach is detected, the system executes the following steps in sequence:

1. The ESP32 identifies which sensor has exceeded its threshold.
2. The buzzer is immediately activated to emit an audible alarm.
3. The relay module is switched ON, activating any connected external safety load.
4. The corresponding LED widget on the Blynk dashboard is updated.
5. A push notification is dispatched to the user's smartphone via Blynk.
6. The event is recorded in the Blynk Cloud event log with a timestamp.
7. The system continues monitoring; the alarm is deactivated once conditions return to normal.

9.4 System Flow Diagram





10. Project Structure

Table 6: Repository Structure

File / Directory	Description
<code>sketch.ino</code>	Main Arduino firmware — sensor reads, Blynk communication, and control logic
<code>diagram.json</code>	Wokwi circuit diagram JSON defining component connections for simulation
<code>libraries.txt</code>	List of required Arduino libraries with version references
<code>README.md</code>	Project overview, setup instructions, and quick-start guide
<code>assets/circuit-diagram.p</code>	Circuit schematic diagram image
<code>assets/dashboard.png</code>	Screenshot of the Blynk IoT mobile dashboard
<code>assets/simulation.png</code>	Screenshot of the Wokwi simulation environment

11. Setup & Deployment Guide

11.1 Prerequisites

- Arduino IDE 2.x or later installed
- ESP32 Board Package added to Arduino IDE (via Boards Manager)

- Blynk IoT account created at blynk.cloud
- Blynk mobile application installed (iOS or Android)
- Blynk template configured with virtual pins V0–V5 and events as defined in Section 7

11.2 Library Installation

Install the following libraries from Arduino IDE
(*Sketch* → *Include Library* → *Manage Libraries*):

- **Blynk** — by Volodymyr Shymanskyy
- **DHTesp** — by beegee-tokyo

11.3 Firmware Configuration

Open `sketch.ino` and update the following credential constants before uploading:

```
#define BLYNK_TEMPLATE_ID      "your_template_id"
#define BLYNK_TEMPLATE_NAME    "your_template_name"
#define BLYNK_AUTH_TOKEN       "your_auth_token"

char ssid[] = "your_wifi_ssid";
char pass[] = "your_wifi_password";
```

Listing 1: Credentials to update in `sketch.ino`

11.4 Flashing the ESP32

1. Connect the ESP32 to your computer via USB.
2. In Arduino IDE, select the correct board:
Tools → *Board* → *ESP32 Dev Module*.
3. Select the correct port: *Tools* → *Port*.
4. Click the **Upload** button.
5. Open the Serial Monitor (115200 baud) to verify Wi-Fi connection and sensor readings.

11.5 Wokwi Simulation

The complete project is available for simulation in the browser via Wokwi, eliminating the need for physical hardware during initial testing and demonstration.

- Navigate to: <https://wokwi.com/projects/466559156265556993>
- Click the green **Play** button to start the simulation.

- Interact with sensors in the simulator to trigger temperature, gas, flame, and motion events.
- Observe the Serial Monitor output and verify Blynk dashboard updates in the Blynk app.

12. Results & Observations

The system was tested both in simulation (Wokwi) and on physical hardware. The following observations were recorded:

- The DHT22 sensor accurately reported temperature and humidity with readings visible on the Blynk dashboard within one to two seconds of change.
- The MQ2 sensor reliably detected gas exposure; the ADC value increased proportionally with gas concentration, triggering the alert at the configured threshold of 3800.
- The flame sensor responded within milliseconds to the introduction of a flame source, immediately activating the buzzer and relay.
- The PIR sensor detected human motion within its specified detection range and sent the corresponding Blynk notification reliably.
- The buzzer and relay deactivated correctly once conditions returned to normal, demonstrating proper hysteresis behavior in the firmware logic.
- Blynk push notifications were delivered to the test smartphone within two to four seconds of a threshold breach, confirming reliable cloud communication over Wi-Fi.
- The Wokwi simulation accurately reproduced all hardware behaviors, validating the firmware logic prior to physical deployment.

13. Applications

This system is directly applicable in the following contexts:

- Residential home safety monitoring for fire, gas, and intrusion detection
- Hostel and dormitory security and environmental monitoring
- Small office or shop security systems
- Industrial storage facility monitoring for gas and fire hazards

- Academic IoT project demonstrations and IEEE paper submissions
- Embedded systems and IoT curriculum laboratory exercises
- Prototype base for commercial smart home security product development

14. Future Enhancements

Table 7: Planned Future Enhancements

Enhancement	Description
GSM Alert Module	Send SMS alerts to the user's mobile number without requiring internet connectivity
IP Camera Integration	Stream live video from the monitored area to the Blynk or mobile app dashboard
AI Threat Detection	Use machine learning models on edge or cloud to classify threat severity from sensor data patterns
Firebase Cloud Storage	Store historical sensor readings and event logs in Google Firebase for long-term analysis
Email Notification	Complement push notifications with automated email alerts via SMTP or SendGrid
Voice Assistant	Integrate with Amazon Alexa or Google Assistant for voice-based status queries
Dedicated Mobile App	Develop a custom Flutter or React Native application for enhanced UI and control
Battery Backup	Add UPS or LiPo battery module to ensure continuous operation during power outages

15. Conclusion

The Smart Home Security System successfully demonstrates the integration of multiple IoT sensors with a cloud-connected microcontroller to deliver a practical, real-time home safety solution. The project achieves all stated objectives: continuous environmental monitoring,

automated hazard detection, audible and relay-based response, and smartphone notification via the Blynk IoT platform.

The ESP32 proved to be a highly capable and cost-effective controller for this application, with its integrated Wi-Fi removing the need for an additional network module. The Blynk platform significantly reduced the development time for cloud connectivity and mobile dashboarding, making it an excellent choice for rapid IoT prototyping.

This project serves as a strong foundation for more advanced home security systems and demonstrates core competencies in embedded systems programming, sensor interfacing, cloud IoT integration, and real-time system design. The techniques and architecture employed here are directly applicable to professional IoT product development.

References

1. Espressif Systems. *ESP32 Technical Reference Manual*. <https://docs.espressif.com>
2. Blynk Inc. *Blynk IoT Documentation*. <https://docs.blynk.io>
3. Aosong Electronics. *DHT22 / AM2302 Datasheet*. Aosong Electronics Co., Ltd.
4. Winsen Electronics. *MQ-2 Gas Sensor Datasheet*. Zhengzhou Winsen Electronics Technology Co., Ltd.
5. Arduino. *Arduino Language Reference*. <https://www.arduino.cc/reference>
6. Wokwi. *ESP32 Simulation Platform*. <https://wokwi.com>
7. Blynk Library for Arduino. GitHub. <https://github.com/blynk/blynk-library>
8. beeg-ee-tokyo. *DHTesp Arduino Library*. GitHub. <https://github.com/beeg-ee-tokyo/DHTesp>